Report 3: Loggy - A logical time logger

Rui Zhang

September 23, 2025

1 Introduction

This report covers the implementation of a logical time logger, which is a mechanism for handling the sequence of events in distributed systems. This system may not have a physically synchronized global clock, so a logical clock allows for global sorting of events from different processes. This is achieved through the so-called Lampert timestamp. The main theme of this study is to gain a deeper understanding of the problems caused by message transmission and how to address them.

2 Main problems and solutions

The main challenge in completing this task is to understand how the Lambert timestamp works. This was mainly done through studying lecture notes. The following is a demonstration of the initial module and its functions.

dist_logger.erl

This module can not be named “logger” due to the built library in Erlang. Therefore, I name it as “dist_logger”, which mainly accepts events and prints them on the screen.

worker.erl

The purpose of the work module is to send and receive events from peers. It also adds a sleep and jitter value to better simulate real networks; The sleep value will determine the activity level of the worker in sending messages, and the jitter value will introduce random delay between sending messages and sending log entries.

test.erl

Set up a simple test to benchmark the system and stop testing after running for 5 seconds (Jitter=200ms).

The above initial code was tested and it can be seen that when the jitter value is set to 200, the messages are printed in the wrong order. However, when the jitter value is set to zero,

the erroneous entries are eliminated.

Wrong order

```
6> test:medium_jitter_test().
=== 测试: 中等Jitter (Jitter=200) ===
Logger started
log: na paul {received,{hello,50}}
log: na john {sending,{hello,50}}
log: na george {received,{hello,61}}
log: na paul {sending,{hello,61}}
log: na ringo {sending,{hello,31}}
log: na ringo {received,{hello,40}}
log: na george {sending,{hello,40}}
log: na george {received,{hello,31}}
log: na ringo {received,{hello,1}}
log: na john {sending,{hello,1}}
log: na ringo {received,{hello,68}}
log: na paul {sending,{hello,68}}
log: na john {received,{hello,76}}
log: na george {sending,{hello,76}}
log: na george {received,{hello,80}}
log: na paul {sending,{hello,80}}
```

Correct order

```
8> test:no_jitter_test().
=== 测试: 无Jitter (Jitter=0) ===
Logger started
log: na john {sending,{hello,50}}
log: na paul {received,{hello,50}}
log: na paul {sending,{hello,61}}
log: na george {received,{hello,61}}
log: na ringo {sending,{hello,31}}
log: na george {received,{hello,31}}
log: na john {sending,{hello,29}}
log: na george {received,{hello,29}}
log: na paul {sending,{hello,11}}
log: na john {received,{hello,11}}
log: na ringo {sending,{hello,48}}
log: na george {received,{hello,48}}
log: na paul {sending,{hello,78}}
log: na ringo {received,{hello,78}}
log: na john {sending,{hello,86}}
log: na paul {received,{hello,86}}
log: na george {sending,{hello,50}}
log: na john {received,{hello,50}}
```

To address further problems, Lamport timestamps were implemented in the initial module.

According to Lamport's definition, each process/worker has its own logical clock, which is represented by a simple counter. In order to capture the so-called sequential occurrence relationship, the following steps were taken: the initial value of the counters for all processes is 0. Whenever a process experiences an event (internal event, sending message, receiving message), it will increase its counter. When a process sends a message, it sends its (incremented) counter value along with the message. When receiving a message, the receiver's counter will be updated to the larger value between their current counter and the timestamp in the received message and then increase by 1. Now I can check the order in which events occurred. If the timestamp of an event is smaller than the timestamps of other events compared to it, it indicates that the event occurred earlier. Lamport timestamp is implemented in a module called time. The time module realizes the following APIs.

zero() --- Provide an initial time value of 0.

inc(Name, T) --- Implement time increment and return T+1.

merge(Ti, Tj) --- Implement Lamport merge rule to take the maximum value of two timestamps.

leq(Ti, Tj) --- Provide time comparison function.

clock(Nodes) --- Create a mapping table to track the latest timestamp of each node.

update(Node, Time, Clock) --- Update the time record of a specific node.

safe(Time, Clock) --- Determine whether the message can be safely output based on the security condition of "timestamp less than or equal to the minimum timestamp of all nodes".

In addition, the hw_logger module has updated a reserved queue for storing log messages that are not yet securely printed. When a new log message arrives, it should update the time, add the message to the hold queue, and then traverse the queue to find the message that can be safely printed now. And events may not happen in the order presented by the log.

The edition without Lamport clock.

```
log: 1 paul {received,{hello,50}}
log: 1 john {sending,{hello,50}}
log: 1 george {received,{hello,61}}
log: 2 paul {sending,{hello,61}}
log: 1 ringo {sending,{hello,31}}
log: 2 ringo {received,{hello,40}}
log: 3 ringo {received,{hello,1}}
log: 2 george {sending,{hello,40}}
log: 3 george {received,{hello,31}}
log: 2 john {sending,{hello,1}}
log: 4 ringo {received,{hello,68}}
log: 3 paul {sending,{hello,68}}
log: 3 john {received,{hello,76}}
log: 4 george {sending,{hello,76}}
log: 5 george {received,{hello,80}}
```

The edition with Lamport clock.

```
Logger started with holdback queue
Holdback Queue: 1
Holdback Queue: 2
Holdback Queue: 3
Holdback Queue: 4
Holdback Queue: 5
log: 1 john {sending,{hello,50}}
log: 1 ringo {sending,{hello,31}}
Holdback Queue: 4
Holdback Queue: 5
Holdback Queue: 6
Holdback Queue: 7
Holdback Queue: 8
log: 2 paul {received,{hello,50}}
log: 2 john {sending,{hello,1}}
Holdback Queue: 7
Holdback Queue: 8
Holdback Queue: 9
log: 3 paul {sending,{hello,61}}
log: 4 paul {sending,{hello,68}}
log: 4 george {received,{hello,61}}
Holdback Queue: 7
Holdback Queue: 8
Holdback Queue: 9
log: 5 george {sending,{hello,40}}
log: 5 paul {sending,{hello,80}}
Holdback Queue: 8
Holdback Queue: 9
log: 6 ringo {received,{hello,40}}
```

Then I change sleep time and jitter time to research the influence on the number of the holdback queue.

sleep/queue/jitter	0ms	20ms	100ms
100ms	22	26	24
600ms	19	14	16
1000ms	12	11	11

According to the results, I found that as the sleep time increases, the number of the holdback queue will decrease.

Bonus

One drawback of using Lamport time is that if the timestamp of one process is higher than another, we still cannot determine whether the event occurred before or after the compared event. However, in our case, it is effective because our program only performs send and receive operations between worker processes. The vector clock solves this problem because it has all the clock values from other processes, not just the values from the sending process. Through this approach, we now know that if the value of one vector is less than all the values of another vector, then the event must have occurred before the other event. This is implemented in a separate module called vect.erl. Its API is exactly the same as the time module, except for its internal presentation.

```
4> clock_comparison:compare_holdback_behavior(1000, 100).
=== Holdback队列行为对比 ===
观察每次消息入队和释放的详细信息

--- Lamport时钟Holdback行为 ---
Logger started with holdback queue
Holdback Queue: 1
Holdback Queue: 2
Holdback Queue: 3
log: 1 john {sending,{hello,50}}
Holdback Queue: 3
Holdback Queue: 4
log: 2 paul {received,{hello,50}}
log: 3 paul {sending,{hello,61}}
Holdback Queue: 3
Holdback Queue: 4
Holdback Queue: 5
log: 4 ringo {received,{hello,61}}
log: 4 paul {sending,{hello,68}}
Holdback Queue: 4
Holdback Queue: 5
```

```
Vector Clock Logger started (initial nodes: [john,paul,ringo])
Note: Logger dynamically handles new nodes without prior knowledge
[Vector QUEUE] Message queued: [john:1,paul:1] from paul, Queue size: 1
[Vector QUEUE] Message queued: [john:1] from john, Queue size: 2
[Vector RELEASE] Released 2 messages, Remaining: 0
log: [john:1] john {sending,{hello,50}}
log: [john:1,paul:1] paul {received,{hello,50}}
[Vector QUEUE] Message queued: [john:1,paul:2,ringo:1] from ringo, Queue size: 1
[Vector QUEUE] Message queued: [john:1,paul:2] from paul, Queue size: 2
[Vector RELEASE] Released 2 messages, Remaining: 0
log: [john:1,paul:2] paul {sending,{hello,61}}
log: [john:1,paul:2,ringo:1] ringo {received,{hello,61}}
[Vector QUEUE] Message queued: [john:2,paul:2,ringo:2] from john, Queue size: 1
[Vector QUEUE] Message queued: [john:1,paul:2,ringo:2] from ringo, Queue size: 2
```

test_dynamic_overlap

```
阶段1: 启动第一组workers (john, paul, ringo)
Vector Clock Logger started (initial nodes: [])
Note: Logger dynamically handles new nodes without prior knowledge
[Vector QUEUE] Message queued: [john:1,ringo:1] from ringo, Queue size: 1
[Vector QUEUE] Message queued: [john:1] from john, Queue size: 2
[Vector RELEASE] Released 2 messages, Remaining: 0
log: [john:1] john {sending,{hello,13}}
log: [john:1,ringo:1] ringo {received,{hello,13}}
[Vector QUEUE] Message queued: [john:1,paul:1,ringo:2] from ringo, Queue size: 1
[Vector QUEUE] Message queued: [john:2,paul:1,ringo:3] from ringo, Queue size: 2
[Vector QUEUE] Message queued: [john:2] from john, Queue size: 3
```

```
阶段2: 动态添加第二组workers (george, paul重叠, pete)
[Vector QUEUE] Message queued: [john:4,paul:6,ringo:5] from paul, Queue size: 1
[Vector RELEASE] Released 1 messages, Remaining: 0
log: [john:4,paul:6,ringo:5] paul {error,
                                   {peers,
                                    [<0.122.0>,<0.124.0>,<0.125.0>,
                                     <0.126.0>]}}
[Vector QUEUE] Message queued: [john:6,paul:7,ringo:10] from paul, Queue size: 1
[Vector QUEUE] Message queued: [john:6,paul:5,ringo:10] from ringo, Queue size: 2
[Vector RELEASE] Released 2 messages, Remaining: 0
log: [john:6,paul:5,ringo:10] ringo {sending,{hello,65}}
log: [john:6,paul:7,ringo:10] paul {received,{hello,65}}
```

The above results indicate that the bonus requirements have been successfully completed.

3 Evaluation

The evaluation of this work is that the order of message printing is incorrect when introducing jitter values which equals to 200 when using only the dist_logger module and worker module. However, when testing with Lampert timestamps, the order of sending and receiving is preserved, but there is a small drawback that messages are printed only after other messages have been sent. The vector clock implementation solves these two problems by printing messages in the correct order and directly printing them after sending. It is observed that if a process/worker is idle and not participating in other processes, no messages will be considered 'safe'.

4 Conclusions

This assignment is simpler than the previous one, but it is definitely not easy. It requires a deep understanding of the Lampert timestamp theory. However, I am grateful that the entire API has been described in detail in the manual, so the process of the program was clear from the beginning. The most difficult part is understanding what secure printing is and what happens when log events with different timestamps are "secure" at the same time. Implementing a vector clock is also a challenge, even though its working mechanism is almost identical to that of a time module. But it is difficult to make different methods work correctly. The main learning topic is to understand the problems that arise when sending messages in distributed systems, as well as a practical understanding of how Lampert timestamps and vector clocks work.